Trabajando de Copiloto con IA

Integrar la IA 🤖 no como un generador de código mágico, sino como una herramienta de copiloto dentro de un proceso estructurado, es la clave para que tus estudiantes aprendan de verdad.

Aquí tienes un resumen de cómo se conectan estos puntos para asegurar el éxito del proyecto:

📑 Flujo de Trabajo para Proyectos JS + IA

Paso	Acción	Objetivo
1. Diagrama UML 📐	Definir clases y relaciones.	Evitar que la IA genere código desorganizado. El UML es el "plano" de la lógica de objetos.
2. Maquetación 🎨	Diseño con herramientas IA.	Separar la estética de la lógica. Obtener un HTML/CSS limpio para luego "darle vida" con JS.
3. Instrucciones Markdown ✍️	Crear el *Prompt Engineering*.	Entrenar a la IA sobre las reglas del proyecto (ej: "Usa solo POO", "No uses `var`", "Aplica `Event Delegation`").
4. Control de Versiones 🌿	Uso de Git/GitHub.	Gestionar el trabajo en equipo y permitir que la IA ayude a revisar ramas (*Pull Requests*).

📄 ENUNCIADO DE LA HACKATHON FRONT 1

🏆 **HACKATHON: “Tienda Virtual con POO”**

🗓️ **Duración sugerida: 4 a 6 horas**

👥 **Equipos: 3 personas**

🧩 **Rol de cada equipo: Asignar los 3 roles principales (Developer, Designer, Product Owner)**

🎯 Objetivo general

Desarrollar una **tienda virtual interactiva** utilizando **HTML, CSS y JavaScript puro**, aplicando **Programación Orientada a Objetos (POO)** y manipulando el **DOM** (selección de elementos, creación dinámica, eventos y actualización en tiempo real).

🛍️ Tema libre

Cada equipo puede elegir el tipo de productos que venderá. Ejemplos:

- Ropa y accesorios
- Tecnología (celulares, auriculares, etc.)
- Librería (libros, cuadernos, mangas)
- Comida rápida o saludable
- Mascotas (juguetes, alimentos)
- Cualquier idea creativa que deseen

✅ Requisitos técnicos obligatorios

1. Programación Orientada a Objetos

Deben crear **al menos dos clases**:

- Producto → con propiedades: id, nombre, precio, imagen (URL o emoji), categoria (opcional pero recomendado).
- Carrito → con propiedades y métodos como:
    - agregarProducto(producto, cantidad)
    - eliminarProducto(id)
    - calcularTotal()
    - vaciarCarrito()
    - renderizar() (muestra los productos del carrito en el DOM)

Opcional pero valorado: clase Tienda o ProductoController para manejar catálogo y filtros.

2. Selección del DOM

Usar métodos como:

- document.querySelector() / getElementById()
- addEventListener para botones, formularios, etc.

3. Creación de elementos dinámicos

- El catálogo de productos debe generarse **desde JavaScript** (no HTML fijo).
- Cada producto del catálogo debe tener un botón "Agregar al carrito".
- El carrito debe mostrar los productos agregados **de forma dinámica** (creando elementos con createElement o innerHTML controlado).

4. Eventos

- Al hacer clic en "Agregar" → se actualiza el carrito y el total.
- Al hacer clic en "Eliminar" (dentro del carrito) → se quita ese producto.
- Botón "Vaciar carrito" → limpia todos los items.
- (Opcional) Filtro por categoría o búsqueda.

5. Estilos CSS

- La interfaz debe ser **responsive** y **agradable visualmente**.
- Usar Flexbox o Grid.
- Se valora la creatividad y coherencia con el tema elegido.

📦 Entregables esperados

1. **Código fuente** (archivos: index.html, styles.css, script.js).
2. **Presentación final (pitch)** de 3 minutos:
    - Nombre de la tienda y tema.
    - Demostración en vivo (funcionando).
    - Explicación de cómo usaron POO y manipulación del DOM.
    - Dificultades encontradas y soluciones.
3. **Repositorio o archivo comprimido** (puede ser en Google Drive, GitHub o enviado por correo).

🧠 Criterios de evaluación (100 puntos)

Criterio	Puntaje
Funcionalidad completa (agregar/quitar/vaciar/total)	25
Uso correcto de POO (clases con métodos, instancias)	20
Manipulación dinámica del DOM (crear elementos, eventos)	20
Diseño y experiencia de usuario (CSS, responsive, claridad)	15
Código limpio y organizado (nombres claros, indentación, comentarios)	10
Originalidad del tema y presentación	10